Bugünkü Akış

4. Bölüm: Docker ile Pratik Kullanım

4.1 Port Yönlendirme (-p, -d, -P)

4.2 Container İçine Girmek (-it)

4.3 docker container exec

4.4 Host ↔ Container Dosya Aktarımı

4.5 inspect ve Çoklu Container'a Giriş

5. Bölüm: Çoklu Container, Hata Ayıklama ve Image Yapısı

5.1 Çoklu Container ve Port Mantığı

5.2 Environment Variable ve Yönetim Araçları

5.3 Hata Ayıklama Komutları

5.4 Port Mantığı Ezberi

5.5 Docker Image'ın Yapısı

6. Bölüm: Image-Container İlişkisi, Docker Hub ve Dockerfile

6.1 Image ile Container Farkı

6.2 Docker Hub ve Registry

6.3 Docker Image Komutları

6.4 Dockerfile ve Komutları

6.5 Build, Cache ve Multi-Stage Dockerfile

7. Bölüm: Docker Network ve Docker Volume

7.1 Docker Network ve CNM

7.2 Network Driver Türleri

7.3 Container Link (Legacy)

7.4 Docker Volume Mantığı

7.5 Named Volume ve Bind Mount

4. Bölüm: Docker ile Pratik Kullanım

4.1 Web Sunucusu Çalıştırma ve Port Yönlendirme

docker container run -p 8080:80 nginx
Bu komutla Docker sırasıyla şunları yapar:

1. Docker Hub'dan nginx image'ını indirir (yerelde yoksa)
2. Bu image'dan yeni container oluşturur
3. Container içinde Nginx'i başlatır
4. Host ile container arasında port bağlantısı kurar

`-p` (publish) parametresi — bu bölümün en önemli konusu:

```
-p 8080:80
   │    │
   │    └── Container'ın dinlediği port
   └─────── Host (bilgisayarın) portu
```

Bilgisayarının 8080 portuna gelen istekler, Docker tarafından container'ın 80 portuna yönlendirilir:

Tarayıcı → localhost:8080 → Docker Engine → Nginx Container (Port 80)
Doğrulama: `http://localhost:8080` açılınca "Welcome to nginx!" sayfası görünür. Bu ekran şunu kanıtlar: Docker Engine çalışıyor, container başarıyla oluştu, port yönlendirmesi doğru yapıldı, tarayıcı container'daki sunucuya erişebiliyor.

4.2 `-d` (Detach) — Arka Planda Çalıştırma

İlk komutta (`-p 8080:80 nginx`) terminal kilitlendi — çünkü Nginx ön planda çalışıyor, logları sürekli terminale yazıyordu. Çözüm:

docker container run -d -p 8080:80 nginx

`-d` sayesinde container arka planda çalışmaya devam eder, terminal serbest kalır, web sunucusu kapanmaz.

Çalışan container'ları listeleme:

docker container ls
→ Container ID, kullanılan image, çalışma durumu, port bilgilerini gösterir.

4.3 `-P` (Büyük P) — Otomatik Port Atama

-p (küçük) → docker container run -d -p 8080:80 nginx → portu SEN belirlersin
-P (büyük) → docker container run -d -P nginx → portu DOCKER otomatik seçer
Docker otomatik seçtiğinde örneğin `32775 → 80` veya `32776 → 80` gibi rastgele boş bir port atanır.

4.4 Linux ve Windows İmajları

Derste iki örnek gösterildi: Linux tarafında **Nginx**, Windows tarafında **Microsoft IIS** image'ı. İkisinin amacı aynı (web sunucusu çalıştırmak), sadece kullanılan işletim sistemi ve web sunucusu farklı.

Ezber akışı:

Image → docker container run -p 8080:80 → Container Oluşur
→ Host Portu → Container Portu → Tarayıcı (localhost:8080) → Nginx Sayfası

4.5 Container İçine Girmek (`-it`)

Soru: "Bu container'ın içine girip Linux komutları çalıştırabilir miyim?" → **Evet.**

Çoğu kişi container'ın sadece bir uygulama olduğunu düşünür, ama aslında içinde küçük bir işletim sistemi vardır:

Docker Host → Container
├── Küçük Linux işletim sistemi
├── Kernel ile iletişim
├── Shell
└── Uygulama
Yani container içinde dosya sistemi vardır, klasör oluşturabilir, paket yükleyebilir, terminal komutları çalıştırabilirsin — görsel arayüzü olmayan küçük bir Linux makinesi gibi.

Girmek için:

```
docker container run -it centos:7
# veya
docker container run --interactive --tty centos:7
```

İki parametre:

- `-i` (Interactive): stdin'i açık tutar, klavyeden yazdığın komutları kabul eder (SSH bağlantısı gibi düşün)
- `-t` (TTY): terminal ekranı oluşturur, normal Linux terminali gibi çalışırsın

Girince prompt değişir:

root@8a31b2c9d4:/#
`root@` → root kullanıcısı, `8a31b2c9d4` → container ID'si. Artık bilgisayarında değil, container'ın içindesin.

İçeride denenen komutlar:

hostname → hostname öğrenildi
cat /etc/os-release → işletim sistemi bilgisi görüldü
mkdir Udemy → klasör oluşturuldu
ls → listelendi
ip addr → IP adresi görüldü
ping google.com → Google'a ping atıldı
Program yükleme: `yum update` ardından `yum install vim` ile Vim editörü kuruldu. Bu, container'ın kapalı bir kutu olmadığını, ihtiyaca göre değiştirilebileceğini gösterdi.

Her image aynı shell'i kullanmaz:

CentOS → bash
Alpine → sh (çok küçük olduğu için bash yoktur)

docker container run -it alpine bash ← HATA (bash yok)
docker container run -it alpine sh ← Doğrusu
Image isminden sonra yazılan her şey container içinde çalışır:

docker container run IMAGE KOMUT
docker container run python:3 python --version
Burada `python:3` = image, `python --version` = container içinde çalıştırılan komut. Komut bilgisayarında değil, container içinde çalışır.

Python container örneği:

docker container run -it python:3
Doğrudan Python ortamına girilir, ardından `a = 5`, `b = 10`, `print(a+b)` gibi kodlar çalıştırılabilir — bilgisayara Python kurmadan!

`--rm` parametresi: Normalde container sistemde kalır. Sadece kısa test için:

docker container run --rm ...

`exit` deyince container otomatik silinir, elle silmeye gerek kalmaz.

Container neden kapanır? Kendisine verilen işi yapar, iş biterse kapanır:

docker container run centos → sürekli görev yok, hemen durur
docker container run centos sleep 30m → 30 dk "uyur", çalışmaya devam eder
Diğer diller: Python image → Python kodları çalıştırıldı; .NET SDK image → console uygulaması oluşturulup çalıştırıldı; OpenJDK image → Java sürümü kontrol edildi. Amaç: her image belirli bir teknoloji için hazır çalışma ortamıdır, bilgisayarına o dili kurmadan kullanabilirsin.

Dockerfile'a giriş: Bu örneklerde paketleri elle kurduk. Gerçek projelerde bunlar tek tek yapılmaz — Dockerfile yazılır. Dockerfile'da hangi image kullanılacak, hangi paketler kurulacak, hangi dosyalar kopyalanacak, hangi komut çalışacak önceden tanımlanır ve image her seferinde aynı şekilde oluşturulur.

Ezber akışı:

Docker Image → docker container run -it IMAGE → Container Oluşur
→ Shell'e Bağlanılır (bash/sh/cmd/PowerShell)
→ Linux/Uygulama Komutları Çalıştırılır
→ Paket Kurulabilir • Dosya Oluşturulabilir • Kod Test Edilebilir
→ exit → Görev biterse Container durur

4.6 `docker container exec` — Çalışan Container'a Bağlanmak

Problem: Nginx/Node.js/Python container'ın zaten çalışıyor. Tekrar `docker container run` yazarsan **yeni** bir container oluşur — oysa istediğin mevcut container'a bağlanmak.

Fark:

docker container run → Yeni container oluşturur (yeni ev inşa etmek)
docker container exec → Var olan çalışan container'a girer (var olan eve kapıdan girmek)
Terminale bağlanmak:

docker container exec -it CentOSContainer bash
`exec` = çalışan container üzerinde işlem yap, `-it` = terminal aç, `CentOSContainer` = hedef container, `bash` = açılacak shell.

Container'a girmeden tek komut çalıştırmak:

docker container exec CentOSContainer mkdir app2
docker container exec CentOSContainer ls
Komut host'ta değil, container içinde çalışır:

Host Bilgisayar → docker exec → Container içinde komut çalışır

Container'ı özelleştirmek: Container oluşturulduktan sonra bile içine yeni paket yüklenebilir. Örnek: `yum install nginx` ile CentOS container'ına Nginx kuruldu. Bu düzenlenmiş container daha sonra yeni bir image olarak kaydedilip tekrar kullanılabilir.

Script çalıştırmak: Tek tek komut yazmak yerine (sistemi güncelle, Vim yükle, klasör oluştur, dosya oluştur gibi adımları içeren) bir script hazırlanıp:

docker container exec ContainerAdi bash script.sh
şeklinde çalıştırılabilir. Gerçek projelerde kurulum işlemleri genelde böyle otomatikleştirilir.

Duran container'a bağlanabilir miyiz? Hayır. `exec` sadece çalışan container'larda işe yarar:

docker container exec -it PythonContainer bash ← container durmuşsa BAĞLANAMAZ
Önce başlatman gerekir:

docker container start PythonContainer
docker container exec -it PythonContainer bash ← şimdi bağlanır
Canlı Nginx sayfasını değiştirme örneği: Nginx container çalıştırılıp `localhost:5080` açıldığında varsayılan sayfa görünüyordu. Sonra container içine girilip varsayılan `index.html` silindi, yeni bir `index.html` oluşturuldu. Tarayıcı yenilenince yeni sayfa göründü. Bu, container çalışırken bile içindeki dosyaların değiştirilebileceğini gösteriyor — ama gerçek projelerde bunun yerine Dockerfile ve volume kullanılır.

Node.js örneği: Node image çalıştırıldı → Node ve npm sürümü kontrol edildi → yeni proje oluşturuldu → `package.json` üretildi → `index.js` yazıldı → uygulama çalıştırıldı → `localhost:3000`'den test edildi. Amaç Node öğretmek değil, hazır bir çalışma ortamını saniyeler içinde kullanabileceğini göstermekti.

Gerçek projelerde nasıl yapılır? Burada yapılanlar öğrenme amaçlıydı. Gerçekte dosyalar elle oluşturulmaz, paketler tek tek kurulmaz, kodlar terminalden yazılmaz — hepsi bir **Dockerfile** içine yazılır ve image oluşturulurken otomatik uygulanır.

Ezber akışı:

Image → docker container run → Çalışan Container
├── docker container exec -it → Container Terminali
└── docker container exec KOMUT → Container içinde tek komut çalışır
Not: SSH, uzak bir makineye ağ üzerinden bağlanmayı sağlayan bir protokoldür. Docker'da container'a erişmek için SSH kurmaya gerek yoktur — `docker container exec` bu işi zaten görür.

4.7 Host ↔ Container Dosya Aktarımı (3 Yöntem)

Problem: "Kodlarım bilgisayarımda, bunları container'a nasıl aktaracağım?"

Host Bilgisayar
├── 1. docker container cp
├── 2. Volume Mount (-v)
└── 3. wget (internetten indirme)

1) `docker container cp` — klasik kopyalama, iki yönlü çalışır

Host → Container:

docker container cp sifre.txt mycontainer:/tmp

Host: şifre.txt → Container: /tmp/şifre.txt

Container → Host:

docker container cp mycontainer:/tmp/index.html .

Buradaki `.` (nokta) Unix/Linux'ta "bulunduğum mevcut klasör" demek. Yani: container içindeki `/tmp/index.html` dosyasını al, terminalde şu an bulunduğum klasöre kopyala.

. → mevcut klasör
.. → bir üst klasör
/Users/beyza/Desktop/ → belirli bir klasör (nokta yerine yazılırsa dosya oraya gider)
Kullanım alanları: config dosyası göndermek, HTML dosyası kopyalamak, log dosyasını dışarı almak.

2) Volume Mount (`-v`) — en çok kullanılan yöntem, bu bölümün en önemli kısmı

Normalde host'taki `index.html` ile container'daki `index.html` **iki farklı dosyadır**. Volume ile host klasörü container'a bağlanır — dosya kopyalanmaz, iki taraf **aynı dosyayı** kullanır.

docker run -v C:\Web:/app nginx (Windows)
docker run -v /Users/beyza/web:/app nginx (Linux)
Artık bilgisayarındaki `web` klasörü, container içinde `/app` olarak görünür.

Avantajı: VS Code'da `index.html`'i kaydedersin, container bunu anında görür — tekrar `cp` yapmana gerek kalmaz. Bu yüzden gerçek projelerde en çok volume kullanılır.

Docker Desktop'ta File Sharing: Mac ve Windows'ta volume bağlayacağın klasörü Settings → File Sharing kısmına eklemen gerekir. Linux'ta genelde buna gerek yoktur.

`-w` (Working Directory): Container açıldığında hangi klasörde başlayacağını belirler:

docker run -w /app
Container açılır açılmaz `/app` klasöründe olursun — özellikle proje klasörlerinde çalışırken kullanışlıdır.

3) `wget` — container internetten kendi indirir

wget https://...

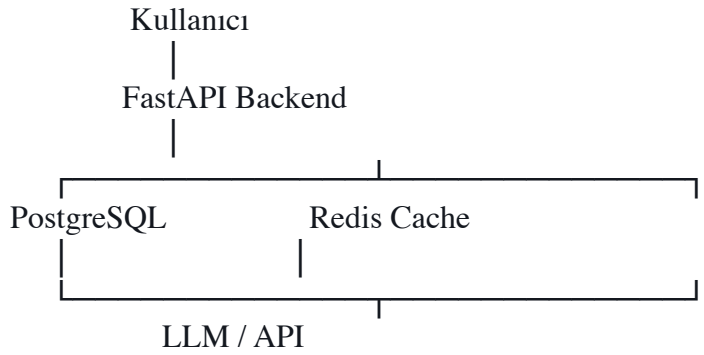
İnternet → Container
Bu yöntemde host bilgisayar devrede değildir; container dosyayı doğrudan internetten indirir.

4.8 `docker container inspect`

Container'ın ayrıntılı bilgilerini (network ayarları, volume bağlantıları, ortam değişkenleri vb.) görüntülemek için kullanılır — Docker açısından öğrenilmesi gereken en önemli komutlardan biri. Windows container'ları ise ayrıca IIS Manager, Server Manager gibi Microsoft araçlarıyla da uzaktan yönetilebilir.

4.9 Çok Container'lı Uygulamalara Giriş

Şimdiye kadar hep **tek** container oluşturduk (sadece Nginx, sadece Python, sadece MariaDB). Ama gerçek projelerde bir uygulama genelde birden fazla servisten oluşur. Örnek — bir AI projesi:



Bunların hepsi **ayrı container** olarak çalışabilir.

5. Bölüm: Çoklu Container, Hata Ayıklama, Yönetim Araçları ve Image Yapısı

5.1 Aynı Anda Birçok Container Çalıştırmak

Videoda aynı bilgisayarda birden fazla servis birlikte çalıştırılıyor: MariaDB 1, MariaDB 2, MSSQL 1, MSSQL 2, Apache, Nginx, ASP.NET. Birbirlerine karışmıyorlar çünkü her biri **ayrı bir container**.

Port mantığı — bu bölümün en kritik noktası:

MariaDB 1: Host 3306 → Container 3306
MariaDB 2: Host 3307 → Container 3306

İkisinin içindeki MariaDB de yine 3306 portunda dinliyor (uygulama bunu değiştirmiyor). Ama **host tarafındaki** portlar farklı olduğu için çakışmıyorlar. Aynı mantık SQL Server, Apache, Nginx için de geçerli.

5.2 Environment Variable (`-e`)

Container oluşturulurken içine ayar/bilgi göndermenin yolu:

-e MYSQL_ROOT_PASSWORD=password
Bu, "root kullanıcısının şifresi bu olsun" bilgisini container'a iletir. Gerçek projelerde (özellikle AI projelerinde) sürekli kullanılır:

OPENAI_API_KEY
DATABASE_URL
SECRET_KEY
JWT_SECRET
REDIS_HOST

5.3 Management Tool ile Bağlanmak

DBForge, SQL Server Management Studio gibi araçlarla container içindeki veritabanına bağlanılıyor. Amaç SQL öğretmek değil — **container içindeki servisin, normal kurulmuş bir servis gibi kullanılabildiğini** göstermek. MariaDB ister bilgisayara kurulsun ister Docker'da çalışsın, bağlanma mantığı aynı.

Bu araçlar olmazsa? `docker container exec` ile container içine girip veritabanının kendi terminali açılır:

Container → MariaDB terminali → SHOW DATABASES / CREATE DATABASE
Grafik arayüz olmasa bile terminalden yönetilebiliyor.

5.4 Database Dosyaları Gerçekte Nerede?

/var/lib/mysql (MariaDB/MySQL)
/var/opt/mssql (MSSQL)
Oluşturulan veritabanları aslında container'ın **dosya sistemi içinde** saklanıyor — database de sonuçta dosyalardan oluşuyor.

Gerçek hayatta: Bunlar tek tek elle yazılmaz, genelde `docker compose up` ile Compose dosyası PostgreSQL, Redis, Backend, Frontend, Nginx gibi servisleri aynı anda ayağa kaldırır. Ama Compose'un arka planda yaptığı, bu bölümde gösterilen işlemlerin ta kendisi.

5.5 Hata Ayıklama Komutları

Container ile ilgili "neden açılmadı, hangi IP'yi aldı, hangi portu kullanıyor, CPU/RAM ne kadar, hata verdi mi" gibi soruların cevabı bu bölümde.

`docker container inspect`

docker container inspect <container_adı>
Container'ın tüm teknik bilgilerini gösterir: ID, oluşturulma zamanı, durumu, IP adresi, gateway, kullanılan image, port eşlemeleri, environment variable'lar, volume/mount bilgileri, çalıştırılan komut.

Kullanım örneği: API çalışmıyorsa ilk bakılacak komutlardan biri budur — yanlış port, bağlanmamış volume, eksik environment variable gibi sorunlar burada görülebilir.

Filtreleme — `grep`:

docker container inspect api | grep IPAddress
docker container inspect api | grep Gateway
docker container inspect api | grep Hostname
`grep`, uzun çıktılar içinde istenen kelimeyi bulmaya yarayan Linux komutu.

`--format`: grep kelime ararken, `--format` doğrudan belirli bir alanı döndürür (örn. sadece IP adresi) — özellikle script yazarken kullanışlı.

`docker container port`

docker container port <container>
Container'ın dışarı açtığı portları gösterir:

80/tcp -> 0.0.0.0:8080
= container içindeki uygulama 80'de çalışıyor, dışarıdan 8080 ile erişiliyor. ⚠️ Container `-p` olmadan oluşturulduysa bu komut boş döner.

`docker container logs`

docker container logs <container>
Bu bölümün en önemli komutlarından biri. Container'ın ürettiği tüm logları gösterir: uygulama açıldı mı, hata verdi mi, database bağlandı mı, API key eksik mi. (Örnek: MySQL oluştururken otomatik atanan root şifresi bile loglarda görünüyor.) Hata ayıklarken ilk bakılacak komut genelde budur.

`docker container top`

docker container top <container>
Container içinde çalışan process'leri listeler — Linux'taki `ps` komutunun Docker versiyonu (örn. `mysqld`, `python`, `nginx`).

`docker container stats`

docker container stats
docker container stats api ← tek container için
Canlı olarak CPU kullanımı, RAM kullanımı, network kullanımı, disk I/O gösterir (örn. `CPU: %8`, `RAM: 350 MB`).

Genel hata ayıklama sırası

docker container ls → inspect → logs → port → top → stats

5.6 Docker'da Uygulama Çalıştırma — Genel İlkeler

- Docker; Python, Node.js, ASP.NET, Java gibi dillerde yazılmış uygulamaları çalıştıran bir ortamdır — hangi dilde yazıldığı önemli değil, önemli olan **uygun image**'ın kullanılması.
- **Sürüm uyumu kritik:** Kod .NET 3.1 kullanıyorsa ama image .NET 2.0 ise uygulama çalışmaz. Runtime sürümleri eşleşmeli.
- Gerçek projelerde uygulamalar **Dockerfile** ile image'a dönüştürülür (ileride detaylı işlenecek).
- **Docker Desktop** üzerinden logs, inspect, stats, start/stop, delete gibi işlemler grafik arayüzle de yapılabilir — terminal karşılıkları zaten öğrenildi.

5.7 Port Mantığı (Ezber Özeti)

Port nedir? Bilgisayardaki uygulamaların kapı numarası. Tarayıcı isteği gönderirken sadece bilgisayarı değil, hangi uygulamaya gideceğini de porttan bilir:

localhost:7860 → Gradio
localhost:8000 → FastAPI
localhost:8080 → Node.js
localhost:80 → Nginx
localhost nedir? Kendi bilgisayarın (127.0.0.1). `http://localhost:8000` = "kendi bilgisayarımdaki 8000 portundaki uygulamaya git."

Docker'da neden iki port var? Container ayrı bir ortam olduğu için hem host'un (Mac/Windows/Linux) hem de container'ın kendi portları vardır — bu yüzden port eşleme (`-p`) yapılır:

docker run -p 1111:8080 node

Tarayıcı: localhost:1111 → Mac'in 1111 portu → Docker → Container'ın 8080 portu → Node.js
1111'i kim belirler? Sen — host tarafındaki portu istediğin gibi seçersin.
8080'i kim belirler? Uygulama — örn. `app.listen(8080)` yazıyorsa Node.js 8080'de çalışır; Docker bunu değiştirmez, sadece yönlendirir.

Herkes istediği portu seçebilir mi? Evet, yeter ki port boş olsun ve 1-65535 aralığında olsun:

-p 3000:8080

-p 5000:8080
-p 9999:8080
hepsi geçerli.

5.8 Python Projesini Volume ile Çalıştırmak

- Bilgisayardaki proje klasörü **volume (mount)** ile container'a bağlanır.
- Container içindeki Python bu dosyaları doğrudan çalıştırır.
- `docker inspect` ile çalıştırılan komut, mount bilgileri, ortam değişkenleri incelenebilir.
- Bu mantık küçük örneklerle sınırlı değil — FastAPI, LangChain, RAG, Gradio gibi gerçek AI projelerinde de aynen kullanılır.

5.9 Docker Yönetim Araçları (GUI)

Docker sadece terminalden değil, grafik arayüzle de yönetilebilir: **Docker Desktop, Portainer, Kitematic (eski)**. Bu araçlar yeni bir teknoloji değil — arka planda yine Docker Engine çalışır, GUI'deki butonlar bizim yerimize `docker run`, `docker ps`, `docker logs`, `docker inspect`, `docker exec`, `docker stop`, `docker rm` gibi komutları çalıştırır.

Portainer: Docker'ı web tarayıcısı üzerinden yönetmeyi sağlayan açık kaynaklı araç. Container oluşturma/başlatma/durdurma/silme, log görüntüleme, container içine bağlanma, CPU/RAM izleme, Image/Volume/Network yönetimi — hepsi grafik arayüzden yapılabilir.

Volume mantığı değişmiyor: Portainer'da da proje klasörü aynı şekilde container'a bağlanır; sadece yönetim şekli değişiyor.

Template (Hazır Şablonlar): Portainer'ın en kullanışlı özelliği — tek tıklamayla Nginx, MySQL, PostgreSQL, MongoDB, MariaDB, WordPress gibi servisler oluşturulabilir, uzun komutlar yazmaya gerek kalmaz.

WordPress örneği neden önemli? WordPress tek başına çalışmaz, bir veritabanına ihtiyaç duyar:

WordPress → MySQL
İki container birlikte çalıştırılıyor — bu, bir sonraki konu olan **Docker Compose**'a giriş niteliğinde.

5.10 Docker Image Nedir? (Derinlemesine)

Docker Image, container oluşturmak için kullanılan hazır ve **değiştirilemeyen (read-only)** bir şablondur:

Docker Image → docker run → Docker Container
Image'ın içinde neler var?

Örnek — Python Image:

Python Image
├── Linux İşletim Sistemi Katmanı
├── Python Runtime
├── Python Kütüphaneleri
├── Ortam Değişkenleri
├── Konfigürasyon Dosyaları
└── Manifest

Neden katmanlardan (layers) oluşur?

Image tek bir büyük dosya değil, üst üste eklenen katmanlardan oluşur:

Layer 4 → Uygulama Dosyaları
Layer 3 → Python Runtime
Layer 2 → Gerekli Kütüphaneler
Layer 1 → Linux İşletim Sistemi

Bu yapı sayesinde Docker aynı katmanı tekrar tekrar indirmez — örneğin iki farklı Python image'ı aynı Linux katmanını paylaşabilir. Bu, disk alanı ve indirme süresinden tasarruf sağlar.

Manifest nedir? Image içindeki küçük dosya; Docker'a hangi katmanların, hangi sırayla indirileceğini ve image'ın sürümünü söyler.

Image read-only'dir

İçeriği doğrudan değiştirilemez. Değişiklik için: yeni bir image oluşturulur ya da Dockerfile kullanılır.

Bir image'dan birden fazla container

Python Image
├── Container 1
├── Container 2
└── Container 3

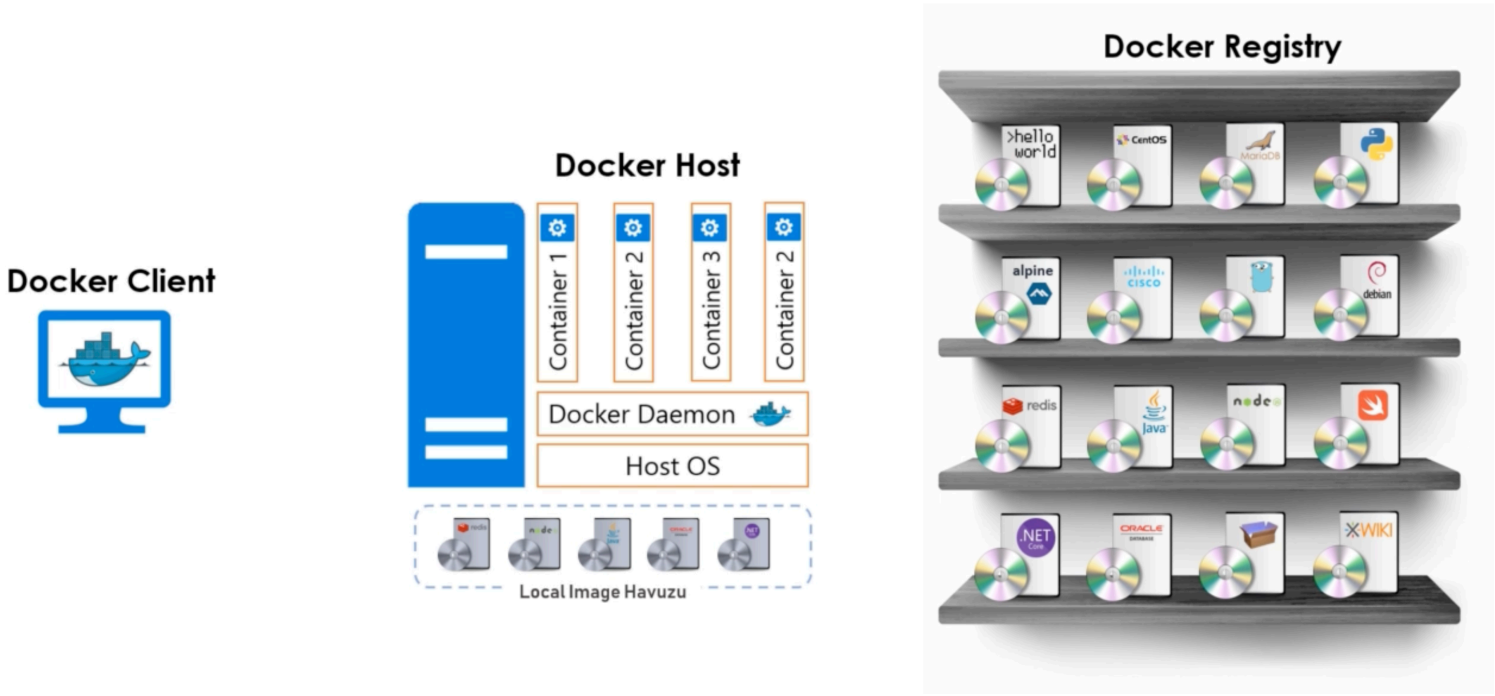
Container silinirse image silinir mi?

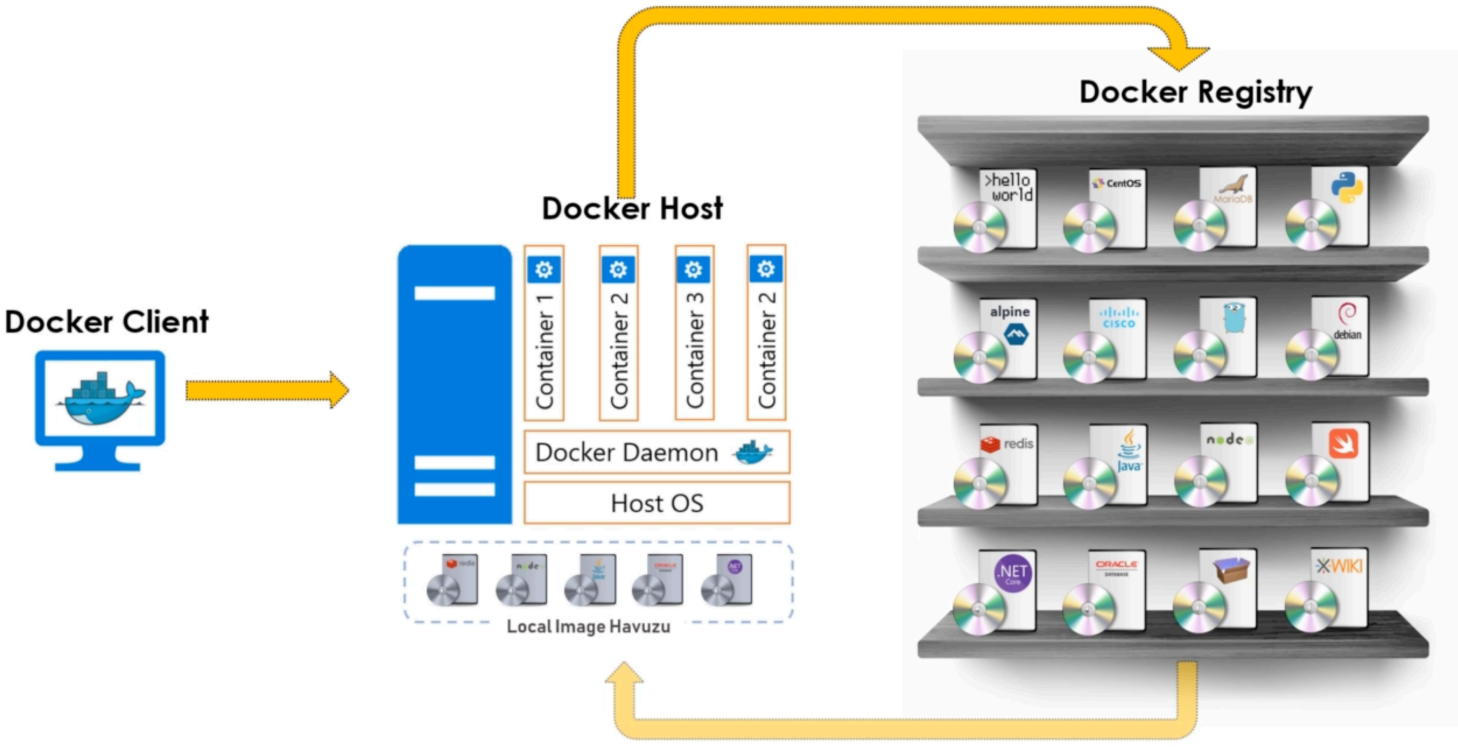
Hayır. `docker run nginx` → nginx image kullanılır → container oluşur. `docker rm container` ile container silinse bile image bilgisayarda kalır; tekrar `docker run nginx` dediğinde Docker image'ı yeniden indirmez.

Image ne zaman silinebilir? Onu kullanan hiçbir container kalmamalı — Docker, çalışan veya duran bir container'ın kullandığı image'ın silinmesine izin vermez. Önce container silinir, sonra image silinebilir.

Dockerfile: Kendi image'ını oluşturmanı sağlayan tarif (recipe) dosyası — bir sonraki bölümde detaylı işlenecek.

Benzetme: Image, sanal makinelerdeki ISO dosyasına benzetilebilir — ISO işletim sistemi kurmanı sağlar, Docker Image container oluşturmanı sağlar. Birebir aynı değil ama ikisi de hazır şablon görevi görür.


Docker Client
Docker Host
Container 1
Container 2
Container 3
Container 2
Docker Daemon
Host OS
Local Image Havuzu
Docker Registry

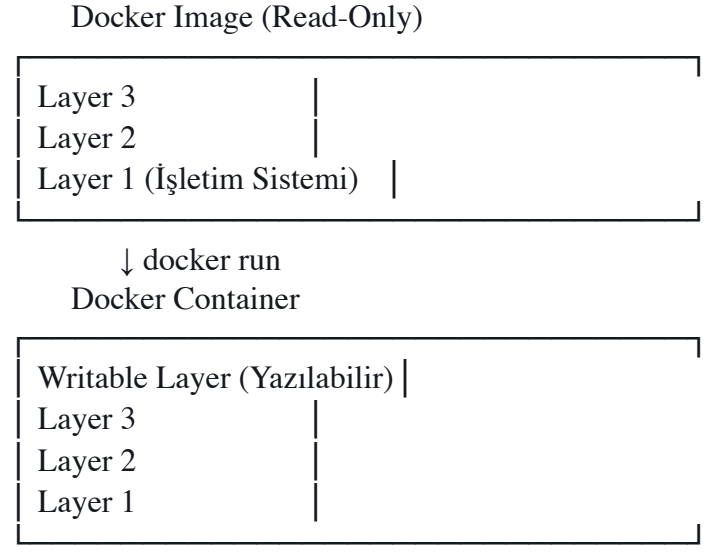

Docker Client
Docker Host
Container 1
Container 2
Container 3
Container 2
Docker Daemon
Host OS
Local Image Havuzu
Docker Registry

6. Bölüm: Image-Container İlişkisi, Docker Hub, Image Komutları ve Dockerfile

6.1 Image ile Container Arasındaki Fark

Bu bölümün en önemli cümlesi:

Container = Image + Yazılabilir (Writable) Katman



Docker, yeni container oluştururken image'ı değiştirmez — sadece en üste yeni bir **yazılabilir katman** ekler.

6.2 Image'ın Katmanlı Yapısı

Örnek — Nginx Image:

Layer 3 → HTML/CSS/JS
Layer 2 → Nginx
Layer 1 → Linux
Her katman bir dosya sistemi değişikliğini temsil eder, oluşturulduktan sonra kilitlenir ve değiştirilemez → bu yüzden Image **read-only**.

6.3 Container'da Yazılan Dosyalar Nereye Gider?

Container içinde `touch test.txt` dersen, bu dosya Image'a **değil**, **Writable Layer**'a yazılır:

Image (Read-Only) → Container (Writable Layer)
Container silinirse (`docker rm`) sadece Writable Layer silinir — **Image olduğu gibi kalır.**

6.4 Image Ne Zaman Silinir?

`docker run nginx` → container oluşur → `docker rm` → container gider ama **Nginx Image bilgisayarda kalır**. Tekrar `docker run nginx` dersen yeniden indirilmez.

6.5 Docker Engine'in Genel Akışı

docker run python
↓
Image bilgisayarda var mı?
Var → direkt Container oluştur
Yok → Docker Hub'dan indir → Container oluştur

6.6 Registry ve Docker Hub

Registry: Image'ların saklandığı depo türü. Docker'ın varsayılan registry'si **Docker Hub**.

Private Registry: Şirketler kendi image'larını herkese açmak istemezse **Docker Trusted Registry** gibi özel çözümler kullanır.

Public → Docker Hub
Private → Şirket Registry'si

6.7 Image İsmi Nasıl Yazılır?

registry/repository:tag
Örnek: `docker.io/python:3.12` → `docker.io` = registry, `python` = repository, `3.12` = tag (versiyon).

Sadece `docker run python` yazarsan Docker otomatik olarak `docker.io/python:latest` şeklinde tamamlar.

6.8 Namespace Nedir?

Docker Hub'da milyonlarca kullanıcı olduğu için isim çakışmasını önlemek için Namespace kullanılır:

mysql → resmi (official) Image
beyzanur/mysql → kullanıcının kendi hesabındaki Image

Format: `KullanıcıAdı/Repository`

6.9 `docker images` Komutu

docker images
Container'dan farklı olarak Image'ların ismi yoktur — **Repository, Tag, Image ID** ile yönetilirler.

Docker Hub

Docker Hub, Image'ların saklandığı, paylaşıldığı ve indirildiği **resmi bulut platformu**.

Official Images: Docker tarafından doğrulanmış, güvenilir ve sürekli güncellenen image'lar — mümkün olduğunca bunlar tercih edilmeli.

Her image sayfasında bulunanlar: nasıl indirilir (`docker pull`), nasıl çalıştırılır (`docker run`), environment variable'lar, Docker Compose örnekleri, desteklenen sürümler (tags), katman bilgileri, dokümantasyon.

Tag (versiyon): `python:3.12`, `python:3.13`, `python:latest` — ihtiyacına göre seçersin.

Repository: Kendi hesabına yüklediğin image'ların saklandığı alan; **Public** (herkes kullanabilir) veya **Private** (izinli kullanıcılar) olabilir.

Namespace: Docker Hub kullanıcı adın aynı zamanda namespace'in — örn. `beyzanur/my-ai-project`.

Docker Image Komutları

Komut	Görev
`docker image ls` / `docker images`	Bilgisayardaki tüm image'ları listeler (Repository, Tag, ID, tarih, boyut)
`docker image pull python:3.12`	Docker Hub'dan image indirir (**container**
`docker image rm python`	Image siler (kullanan container varsa izin
`docker image inspect python`	Katmanlar, hash değerleri, ortam değişkenleri,
`docker login` / `docker logout`	Docker Hub'a giriş/çıkış
`docker image push`	Yerel image'ı Docker Hub hesabına yükler
`docker container commit containerAdi yeniImage`	Çalışan bir container'ın halini yeni image olarak kaydeder
`docker image tag eskiImage`	Image'a yeni isim/sürüm verir (kopyalamaz,
`docker image history python`	Image'ın hangi katmanlardan oluştuğunu
`docker search python`	Docker Hub'da image arar (isim, yıldız sayısı,

Image'ı Docker Hub'a gönderme süreci:

Container → docker container commit → Image Oluştur
→ docker image tag → docker login → docker image push → Docker Hub
Not: Gerçek projelerde `commit` yerine genellikle **Dockerfile → docker build → docker push** akışı kullanılır. `commit` daha çok mantığı öğrenmek ve hızlı prototip için.

Dockerfile

6.10 Dockerfile Nedir?

Bir Docker Image'ının nasıl oluşturulacağını **adım adım tanımlayan metin dosyası**. İçinde: hangi base image kullanılacak, hangi klasörler oluşturulacak, hangi dosyalar kopyalanacak, hangi paketler kurulacak, uygulama nasıl çalıştırılacak — hepsi satır satır yazılır.

Neden kullanılır? `docker run` + container'a girip elle dosya kopyalama/paket kurma yerine, bu işlemler tek dosyada tanımlanır ve otomatik gerçekleşir → aynı image her zaman aynı şekilde tekrar üretilebilir.

Çalışma mantığı: Satır satır okunur, her satır bir Layer oluşturur, son satırda tüm katmanlar birleşip Image oluşur.

Büyük projelerde: Her servis (Backend, Frontend, Database, Redis...) için ayrı Dockerfile hazırlanır. Bunları tek tek çalıştırmak zahmetli olduğu için **Docker Compose** devreye girer — tüm servisleri tek komutla ayağa kaldırır.

Uygulama Kodu → Dockerfile → docker build → Docker Image → docker run → Container

6.11 Dockerfile Komutları

Komut	Görev
`FROM python:3.12`	Temel (base) image'ı belirler — genelde ilk satır
`RUN pip install -r requirements.txt`	Image oluşturulurken çalıştırılır (her RUN yeni bir katman oluşturur, birleştirmek daha verimli)
`ADD app.zip /app`	Host'tan **veya internetten** dosya kopyalar
`COPY . /app`	Sadece host'taki dosyaları kopyalar (günlük kullanımda tercih
`WORKDIR /app`	Container içindeki çalışma dizinini belirler
`VOLUME /data`	Kalıcı veri alanı oluşturur (container silinse bile veriler korunur)
`EXPOSE 8000`	"Bu uygulama 8000'de çalışıyor" bilgisini ekler (portu açmaz, sadece bildirir; açmak için `docker run -p` gerekir)
`LABEL`	Image'a açıklama/metadata ekler (çalışmayı etkilemez)
`USER appuser`	İşlemlerin hangi kullanıcıyla çalışacağını belirler (güvenlik)

`ENV`	Ortam değişkeni tanımlar
`CMD`	Container başlarken çalışacak varsayılan komut (yalnızca son CMD...)
`ENTRYPOINT`	Container'ın ana başlangıç komutu — CMD ile birleşir (esnek)

RUN vs ADD/COPY farkı: ADD internetten de indirebilir, COPY sadece yerel dosyaları taşır.

Shell Form vs Exec (JSON) Form:

Shell: CMD python app.py
Exec: CMD ["python","app.py"] ← daha güvenli, önerilen
Örnek akış:

FROM python:3.12 → RUN paketleri yükle → WORKDIR /app
→ COPY requirements.txt → RUN pip install → COPY proje dosyaları → CMD python main.py

6.12 Dockerfile'dan Image Oluşturma (Build)

Dockerfile → docker image build → Docker Image → docker run → Container
Temel komut:

docker image build -t image_adi .
`build` = derler, `-t` = isim verir, `.` = Dockerfile'ın bulunduğu klasör.

Farklı klasördeyse:

docker image build -f /dosya/yolu/Dockerfile -t image_adi .
Build sırasında: Dockerfile satır satır okunur (FROM, RUN, COPY, CMD...), her satır yeni bir katman oluşturur, katmanlar birleşip yeni image olur.

Build Cache

İlk build'de tüm satırlar çalışır. İkinci build'de aynı Dockerfile kullanılırsa `Using cache` mesajı görülür — Docker daha önce oluşturduğu katmanları tekrar kullanır, build çok daha hızlı olur.

Cache ne zaman kullanılmaz? Dockerfile'da bir satır değişirse (örn. COPY), **o satırdan sonraki** katmanlar yeniden oluşturulur; öncekiler cache'den gelir.

Cache'i devre dışı bırakmak:

docker image build --no-cache
`.dockerignore`

Container'a kopyalanmasını istemediğin dosyaları belirtir (log dosyaları, geçici dosyalar, `.git` klasörü, gizli bilgiler). Böylece: image küçülür, build hızlanır, gizli dosyalar yanlışlıkla eklenmez.

6.13 RUN, CMD ve ENTRYPOINT Farkı

	Ne zaman çalışır	Amaç
RUN	`docker build` sırasında	Paket kurulumu, sistem hazırlığı (her RUN yeni katman → mümkün olduğunca az kullan)
CMD	`docker run`	Varsayılan komut, dışarıdan değiştirilebilir
ENTRYPOINT	`docker run` sırasında	Ana başlangıç komutu, CMD ile birleşir

En yaygın kombinasyon:

ENTRYPOINT ["python"]
CMD ["main.py"]
→ Container her zaman Python ile başlar, varsayılan `main.py` çalışır. Kullanıcı `docker run image test.py` derse çalışan komut `python test.py` olur — ENTRYPOINT sabit kalır, CMD değişir.

6.14 Farklı Teknolojilerde Aynı Mantık

Python, Node.js, .NET, C, Web Sunucusu — hepsinde aynı akış:

Base Image seç → Paket kur → Çalışma dizini belirle → Dosyaları kopyala → Bağımlılıkları yükle → Uygulamayı çalıştır
Gereksiz RUN komutlarından kaçınmak → daha az katman → daha küçük/performanslı image.

6.15 Multi-Stage Dockerfile

Problem: Tek aşamalı Dockerfile'da derleyici, paketler, geçici dosyalar hep Image içinde kalır → **büyük image**. Büyük image = geç indirilir, geç yüklenir, fazla disk kullanır, güvenlik riski artar, yavaş çalışabilir.

Image küçültme yöntemleri

1. `.dockerignore` kullan
2. Gereksiz paket kurma
3. RUN komutlarını birleştir (`apt update && apt install -y nginx git`)
4. Küçük base image kullan (`FROM alpine`)
5. **Multi-Stage Dockerfile kullan** ← bu bölümün asıl konusu

Mantığı

Tek aşamalı: Kod → Derleme → Compiler → Kütüphaneler → Çalıştırma → hepsi aynı Image'da kalır.

Multi-Stage — iki ayrı image:

1. Stage (Builder): Kod → Compiler → Build → Executable oluşur
2. Stage (Final): Sadece Executable alınır → Yeni küçük Image → CMD
İkinci image'da compiler, gcc, build araçları, geçici dosyalar **yok** — sadece çalışan program var.

Yazımı

FROM alpine AS builder
(kod gelir, derlenir, çıktı oluşur)

FROM alpine
COPY --from=builder ... (ilk aşamadan sadece gerekli dosya alınır)
CMD ...

Scratch Image

FROM scratch
Tamamen boş image — Linux yok, paket yok, shell yok. Uygulaman hiçbir şeye bağımlı değilse kullanılabilir; en küçük image türüdür. Ama bağımlılığın varsa scratch kullanılamaz.

Python Örneği (3 aşama)

Base → Python Image
Builder → paketleri kur, projeyi hazırla
Final → builder'dan gerekli dosyaları al, CMD ile çalıştır

C Programı Örneği — en çarpıcı kanıt

Normal Dockerfile: 178 MB
Multi-Stage Dockerfile: 4.22 MB
Sadece Dockerfile yapısı değiştirilerek image **178 MB'dan 4.22 MB'a** düştü, uygulama sorunsuz çalıştı.

6.16 İyi Bir Dockerfile Yazma Kuralları (Best Practices)

1. **Komut sırasına dikkat et** — sık değişen dosyaları (`COPY . .`) en sona koy, önce paket kurulumunu yap. Böylece cache verimli kullanılır.
2. **Gereksiz dosya kopyalama** — `COPY . .` yerine sadece gerekli dosyaları kopyala (`COPY app.jar .`)
3. **RUN komutlarını birleştir** — az katman = küçük image
4. **Gereksiz paket kurma** — sadece ihtiyaç duyulanı yükle
5. **Paket cache'ini temizle** — aynı RUN içinde `apt clean` gibi
6. **Resmî (Official) Image kullan** — Python, Node.js, Nginx gibi hazır ve güvenilir image'lar
7. **`latest` etiketini kullanma** — `python:latest` yerine `python:3.12` gibi belirli sürüm kullan (latest zamanla değişip uygulamanı bozabilir)
8. **Küçük base image tercih et** — mümkünse `alpine`
9. **Bağımlılıkları açıkça belirt** — örn. Maven projesi için Maven image

10. **Multi-Stage Dockerfile kullan** — derleme araçları son image'a eklenmesin

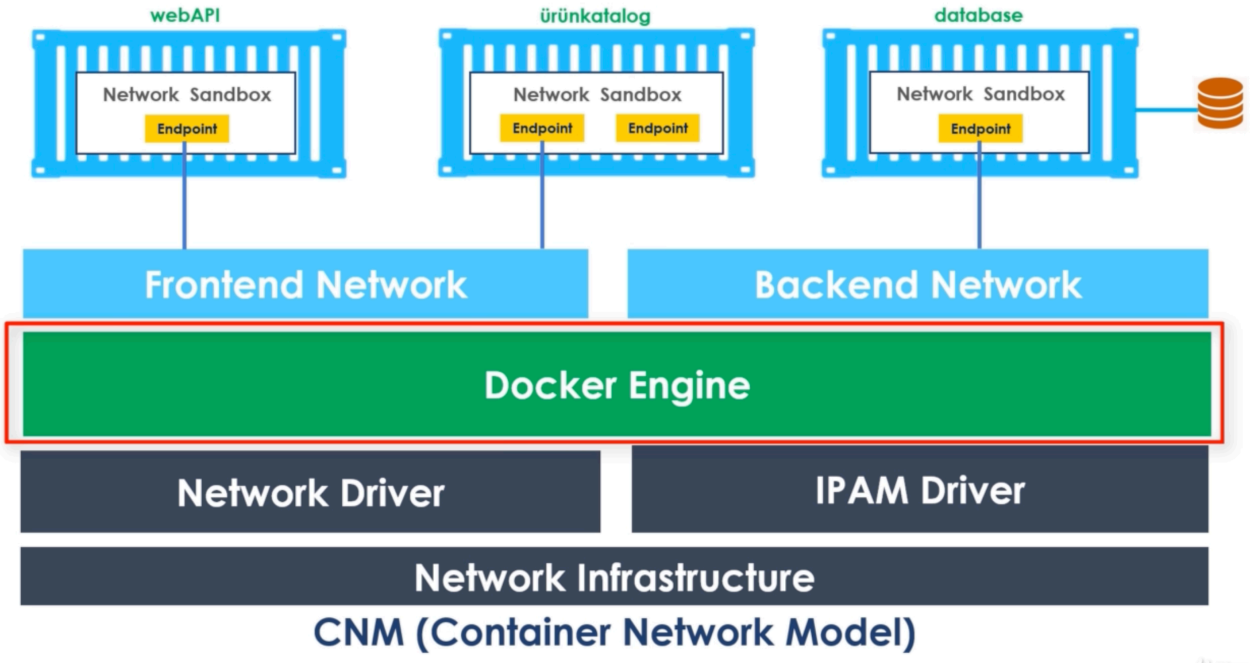
6.17 Python Projesini Docker'a Aktarma (Uygulamalı Örnek)

Proje: `server.py` (Flask, port 5000) + `requirements.txt`

Normal Dockerfile akışı:

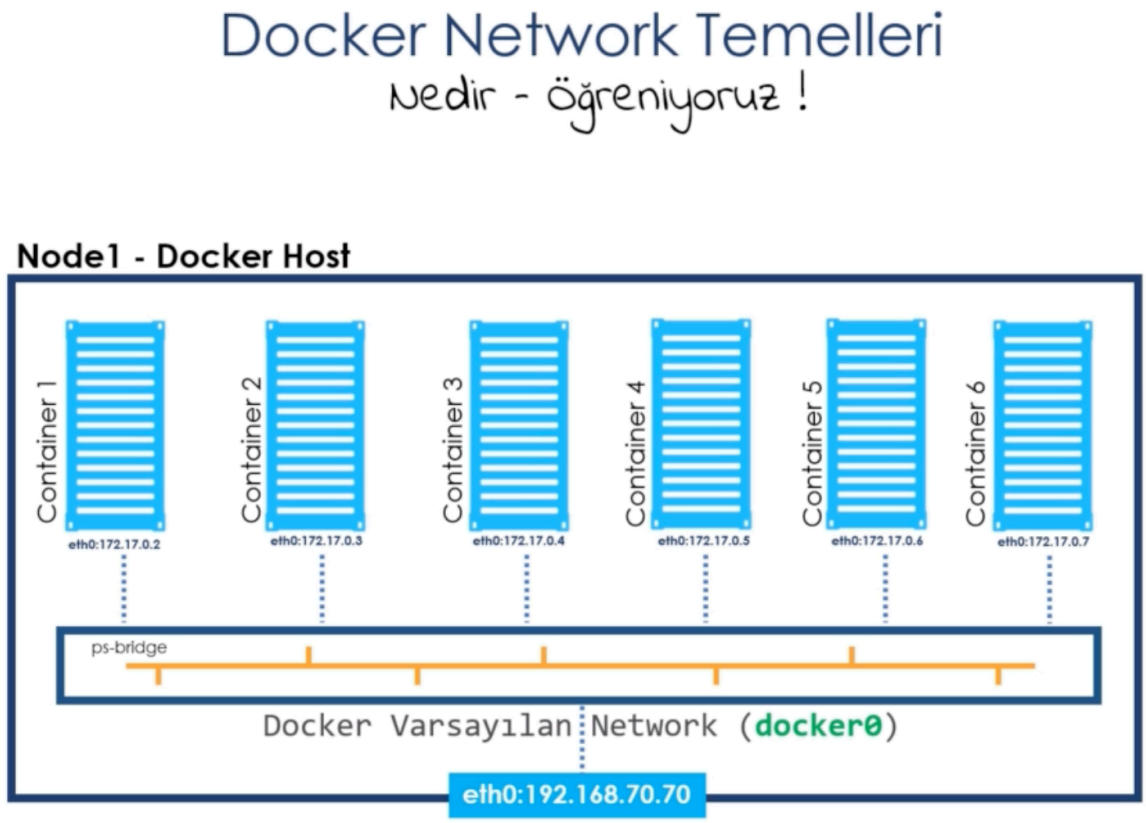
Python 3.8 base → WORKDIR /app → COPY requirements.txt
→ pip install -r requirements.txt → COPY server.py → CMD (uygulamayı başlat)
→ build → run → localhost:5000 üzerinden doğrulama
Multi-Stage versiyonu:

Builder Stage: Python 3.8 → requirements.txt kopyala → bağımlılıkları yükle
Final Stage: Python Slim (küçük) → COPY --from=builder ile bağımlılıkları al
→ proje dosyalarını kopyala → PATH ayarla → CMD ile başlat
Neden Multi-Stage? Normal Dockerfile'da derleme dosyaları, gereksiz bağımlılıklar, geçici dosyalar image'da kalır. Multi-Stage'de sadece gerekli bağımlılıklar alınır → daha küçük, daha



güvenli image.

Python Projesi → Dockerfile → Builder Stage (Python + requirements)
→ Final Stage (Python Slim + gerekli dosyalar)
→ Docker Image → Container → Python Web Uygulaması
Böylece uygulama aynı şekilde çalışırken daha küçük, güvenli ve performanslı bir Docker Image elde edilmiştir.



7. Bölüm: Docker Network ve Docker Volume

7.1 Docker Network Nedir?

Container'ların birbirleriyle ve dış dünyayla nasıl iletişim kuracağını yöneten ağ yapısı. Tek container'da pek gerek yok, ama gerçek uygulamalarda (Web, API, Database, Redis, Nginx) birçok container birlikte çalışır ve birbirini bulabilmesi gerekir.

Her container'a otomatik IP verilir:

Container A → 172.17.0.2
Container B → 172.17.0.3
Container C → 172.17.0.4

7.2 docker0 (Varsayılan Bridge Network)

Docker kurulunca otomatik olarak `docker0` adında bir **Bridge Network** oluşur. Sen network belirtmezsen tüm yeni container'lar buraya bağlanır:

Container → docker0 (Bridge) → Diğer Container'lar

7.3 Container Network Model (CNM)

Docker'ın ağ standardı. Amacı: container'ları bağlamak, farklı network'ler oluşturmak, bir container'ı birden fazla network'e bağlayabilmek, trafiği yönetmek.

3 temel bileşen:

Bileşe	Görev
Sand box	Container'ın ağ ayarlarının tutulduğu alan (IP, DNS, routing tabloları). Her container'ın kendi Sandbox'ı var → izolasyon sağlar
Endp oint	Container'ın network'e bağlandığı nokta. Bir Endpoint sadece **tek** network'e bağlanabilir
Netw	Endpoint'lerin oluşturduğu iletişim ağı — aynı network'teki container'lar birbiriyle

Container → Endpoint → Network

7.4 Bir Container Birden Fazla Network'e Bağlanabilir mi?

Evet:

Frontend Network → Web API → Backend Network → Database
Web API iki ağa birden bağlı — hem frontend'den istek alır, hem backend üzerinden Database'e erişir.

7.5 Diğer Bileşenler

- **Network Driver:** network oluşturma/silme, container ekleme/çıkarma işlerini yönetir.
- **IPAM (IP Address Management):** IP havuzu oluşturur, container'a IP atar, gerekirse geri alır.
- **Libnetwork:** Docker'ın ağ altyapısını sağlayan kütüphane — Docker ile Network Driver arasındaki bağlantıyı kurar.

Docker Engine
├── Network Driver
└── IPAM Driver
↓
Docker Network
├── Container A (Sandbox, Endpoint)
├── Container B (Sandbox, Endpoint)
└── Container C (Sandbox, Endpoint)

7.6 Docker Network Komutları

Komut	Görev
`docker network`	Tüm network komutlarını listeler
`docker network create my-`	Yeni network oluşturur

`docker network ls`	Mevcut network'leri listeler (varsayılan: bridge, host, none)
`docker run --network my-network nginx`	Container'ı doğrudan belirli network'e bağlar
`docker network inspect`	Network ID, Driver, Subnet, bağlı container'lar,
`docker network connect`	Çalışan container'ı sonradan başka bir network'e
`docker network disconnect`	Container'ı network'ten çıkarır
`docker network rm`	Network siler (içinde container varsa önce

Ana mesaj: Tüm servisleri aynı network'e koymak zorunda değilsin — sadece iletişim kurması gerekenleri aynı network'e bağlayarak hem güvenlik hem yönetilebilirlik kazanırsın.

7.7 Network Driver Türleri

1) Bridge Network (varsayılan, en çok kullanılan)

Docker kurulunca otomatik oluşan `docker0`. Belirtilmezse tüm container'lar buraya bağlanır.

Gerçek proje örneği — izolasyon amaçlı kullanım:

Frontend Network: Web API, Product Catalog
Backend Network: Product Catalog, Database
Product Catalog iki ağa da bağlı (hem Web API ile hem Database ile konuşuyor), ama **Web API doğrudan Database'i göremiyor** — güvenlik bu şekilde sağlanıyor. Yani Bridge sadece haberleşme değil, izolasyon için de kullanılıyor.

2) Host Network

Container'ın kendi ağı olmaz, doğrudan bilgisayarın ağını kullanır:

Bilgisayar IP: 192.168.1.10 → Container IP: 192.168.1.10 (aynı)
Avantaj: yüksek performans. **Dezavantaj:** izolasyon/güvenlik yok → production'da önerilmez.

3) None Network

Container hiçbir ağa bağlanmaz, sadece `127.0.0.1` var. İnternete çıkamaz, başka container göremez. **En güvenli model.**

4) Overlay Network

Bridge tek bilgisayar içindir. İki farklı sunucudaki (Host A'daki Database, Host B'deki Application) container'lar birbirini bulması gerekiyorsa Overlay kullanılır — sanki aynı ağdaymış gibi çalışırlar. **Docker Swarm** bunun üzerine kuruludur.

5) Macvlan

Container'a gerçek bir MAC adresi verilir; ağda normal bir fiziksel cihaz gibi görünür (laptop, yazıcı, kamera gibi). Eski ağ sistemleriyle çalışırken kullanılır.

7.8 Container Link (Legacy — artık kullanılmıyor)

Problem (eskiden): Docker'ın DNS sistemi yoktu. Web uygulaması Database'e IP ile (`172.17.0.2`) bağlanıyordu, ama Database container'ı yeniden oluşunca IP değişiyordu (`172.17.0.8`), bağlantı kopuyordu.

Eski çözüm — `--link`:

docker run --link database web
Docker, Web container'ının `/etc/hosts` dosyasına `172.17.0.2 database` satırı ekliyordu — artık `ping database` çalışıyordu, isim otomatik IP'ye çevriliyordu.

Örnek: Redis oluşturulup, BusyBox `--link redis` ile bağlandığında BusyBox içinden `ping redis` çalışıyordu.

Bugün neden gerekli değil? Docker'ın kendi **dahili DNS sistemi** aynı bridge network'teki container'lar arasında ismi otomatik IP'ye çeviriyor. Artık `ping database` demen yeterli — `--link` yazmana gerek yok. `--link` **legacy** kabul ediliyor, yeni projelerde önerilmiyor.

Docker Network Driver Seç → Container'lar oluşturulur
→ Aynı network'telerse haberleşebilir
→ Eski Docker: --link ile isim çözümleme
→ Yeni Docker: Docker DNS otomatik yapar

Docker Volume

7.9 Docker Volume Neden Gerekli?

Normal akış:

Image → Container → Dosyalar
Container silinince (`docker container rm`) **içindeki tüm dosyalar da silinir** — çünkü container'ın dosya sistemi **geçici (ephemeral storage)**.

Neden büyük problem? Bir e-ticaret sitesinde container içinde siparişler, müşteriler, ürünler, loglar tutuluyorsa; container çöküp yenisi oluşturulduğunda **tüm bu veriler kaybolur.** Gerçek hayatta kabul edilemez.

7.10 Çözüm: Docker Volume

Container → Docker Volume → Host Disk

Veri artık container'ın içinde değil, container'dan bağımsız bir disk alanında. Container silinse bile **Volume kalır** — yeni container aynı Volume'a bağlanınca eski veriler geri gelir.

7.11 İki Depolama Türü

Tür	Yer	Container silinince
Ephemeral (geçici)	Container'ın Writable Layer'ı	Silinir
Persistent (kalıcı)	Docker Volume → Disk	Kalır

Stop/Start vs Delete farkı:

```
docker stop web  → container kapanır, dosyalar durur
docker start web → her şey aynen devam eder
docker rm web    → container tamamen silinir, Volume yoksa veriler de gider
```

7.12 Volume Nerede Saklanır?

```
Linux:   /var/lib/docker/volumes
Windows: C:\ProgramData\Docker\volumes
```

7.13 Aynı Volume Birden Fazla Container Tarafından Kullanılabilir

```
Volume
  ├── Web (yazar)
  └── Backup (okur)
```

7.14 Storage Driver

Container'ın Writable Layer'ını yöneten mekanizma — "container'ın dosyalarını host diskine nasıl yazacağını" belirler. En yaygın: **Overlay2** (Linux'ta en iyi performans, varsayılan).

```
Image (Read-Only) → Writable Layer  ← Storage Driver bunu yönetir
```

7.15 İki Volume Bağlama Yöntemi

	Kim yönetir	Kullanım alanı
Named Volume	Docker kendi yönetir (`/var/lib/docker/volumes`)	En güvenli, en önerilen — kalıcı uygulama verileri
Bind Mount	Sen kendi klasörünü seçersin	Geliştirme ortamında yaygın — kaynak kod, config dosyaları

7.16 Docker Volume Komutları

```
docker volume create depo          → Volume oluşturur
```

docker volume ls → Volume'ları listeler
docker volume inspect depo → Detayları gösterir (Mountpoint dahil)
docker volume rm depo → Volume siler
docker volume prune → Kullanılmayan tüm Volume'ları temizler

7.17 Volume'u Container'a Bağlamak

docker run -d --name test1 -v depo:/depo nginx
`depo` = Host tarafındaki Volume, `/depo` = Container içindeki klasör. Container `/depo` içine yazdığı her şey aslında Host'taki Volume'a yazılır.

Uygulamalı akış (eğitmenin demosu):

1. Host'ta Volume'a bazı dosyalar konuluyor (Muhasebe, Satış, Liste)
2. Volume container'a bağlanıyor → `cd /depo` diyince aynı dosyalar container içinde de görünüyor (aynı fiziksel klasör)
3. Container içinde `rapor.txt` oluşturuluyor → Host tarafındaki Volume klasöründe de görünüyor
4. `docker rm test1` ile container siliniyor → **Volume'daki dosyalar hâlâ duruyor** ✅
5. Aynı Volume yeni bir container'a bağlanıyor → eski dosyaların hepsi geri geliyor

7.18 Docker'ın Otomatik Volume Oluşturması

docker run -v depoYedek:/backup
`depoYedek` isimli Volume yoksa Docker onu **otomatik oluşturur** — önceden `docker volume create` yazman şart değil.

7.19 Read Only Volume (`:ro`)

-v depo:/depo:ro
Container: dosyaları okuyabilir , ama yeni dosya oluşturamaz/silemez .

docker volume create depo → Host'ta oluşur
↓
docker run -v depo:/depo → Container'a bağlanır
↓
Container'ın /depo'ya yazdığı her şey Host'a kaydedilir
↓
Container silinse bile Volume kalır
↓
Yeni container aynı Volume'u bağlayınca eski verilere erişir

7.20 Bind Mount

Docker'ın kendi Volume'u yerine **bilgisayarındaki gerçek bir klasörü** doğrudan container'a bağlama:

Host: ortak-paylasim → Container: /depo

`-v` ile:

```
docker run -it --name depo-bagli-1 \
  -v "$(pwd)/ortak-paylasim:/depo" \
  busybox
```

`$(pwd)` = terminaldeki mevcut klasörün tam yolu. Host'ta oluşturulan dosya container'da görünür, container'da oluşturulan dosya host'ta görünür.

`--mount` ile (daha açık/okunabilir):

```
docker run -it --name depo-bagli-2 \
  --mount type=bind,source="$(pwd)/ortak-paylasim",target=/depo \
  busybox
```

`-v` ile `--mount` arasındaki fark:

- `-v`: kaynak klasör yoksa Docker **otomatik oluşturur**
- `--mount`: kaynak klasör yoksa Docker **hata verir**

Read-only bind mount:

```
-v "$(pwd)/ortak-paylasim:/depo:ro"
```

Named Volume vs Bind Mount: Bind Mount genelde Dockerfile içine yazılmaz (host'taki yol her bilgisayarda farklı olabilir) — `docker run` veya Compose sırasında tanımlanır. Named Volume kalıcı uygulama verileri için, Bind Mount kaynak kod/config gibi host'tan doğrudan erişilmesi gereken içerikler için tercih edilir.

7.21 Dockerfile İçinde Volume Kullanımı (`VOLUME` komutu)

dockerfile

```
FROM centos
VOLUME ["/mountpoint-demo"]
```

Bu, bu image'dan oluşturulan **her container'da** `/mountpoint-demo` klasörünün otomatik olarak bir Docker Volume'a bağlanacağını belirtir.

Akış:

```
docker build -t mountpoint-image .
      ↓
docker run  (herhangi bir -v verilmese bile)
      ↓
Docker otomatik Volume oluşturur ve /mountpoint-demo'ya bağlar
```

`docker image inspect` ile image içinde bu Volume tanımı görülebilir. Container'da `/mountpoint-demo` içine `test.txt`, `oku.txt` gibi dosyalar oluşturulduğunda, `docker`

`volume ls` ve `docker inspect` ile Volume'un gerçekten host'ta oluştuğu ve dosyaların orada saklandığı doğrulanabilir.

`docker inspect` çıktısındaki:

- **Source** → Host'taki gerçek Volume yolu
- **Destination** → Container içindeki bağlandığı klasör

Dockerfile'daki VOLUME ile `docker run -v` farkı: VOLUME, image seviyesinde tanımlanır ve o image'dan oluşan **her** container'a otomatik uygulanır; `docker run -v` ise her seferinde manuel belirtilir.

Bu bölüm Network ve Volume diye iki büyük ana konuyu kapsıyordu — network driver türleri, CNM bileşenleri, link mekanizması, sonra ephemeral/persistent depolama, named volume ve bind mount farkları. Bu, Docker'ın "üretim seviyesi" konularının neredeyse tamamı sayılır.